

MNIST_final

June 13, 2026

1 MNIST Bandit Classification — Clean Comparison

Five methods compared on MNIST under **bandit feedback** (binary 0/1 reward per sampled action):

Method	Description
ce	Cross-entropy with full labels (oracle upper bound)
pg	Policy Gradient (REINFORCE + expected-reward baseline)
dg	Delightful PG (sigmoid-gated PG, $\beta=1$)
entropy_hv	PG + exact entropy bonus: $\text{loss} = \text{PG} - \beta \cdot H(\cdot)$
entropy_lv	PG + score-function entropy estimator (DistG-style)

Protocol - Data: 59 000 train / 1 000 val / 10 000 test - Hyperparameter sweep: 1 seed; pick best config per method by **final val accuracy** - Final run: 5 seeds with best config; report mean $\pm$ stderr

```
[1]: import math
import itertools
import torch
import matplotlib.pyplot as plt

from data import load_mnist_split
from train import train_run

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Device: {DEVICE}")
```

Device: cpu

```
[2]: # Global seeds
DATA_SEED = 42 # fixes the 59k/1k train-val split (never changes)
SWEEP_SEED = 0 # single seed used during hparam sweep
FINAL_SEEDS = list(range(5)) # seeds for the final multi-seed run

# Shared training config (matches original MNIST.ipynb)
BASE_CFG = dict(
    hidden_sizes = (50, 50),
    batch_size = 100,
```

```

train_steps      = 10_000,
eval_every       = 200,
eta              = 1.0,          # DG temperature (fixed)
baseline_mode    = "expected",
samples_per_image= 1,
device           = DEVICE,
)

# Hyperparameter grids
LR_VALUES        = [1e-4, 3e-4, 1e-3, 3e-3, 1e-2]
BETA_VALUES      = [1e-2, 3e-2, 1e-1, 3e-1, 1e0]

METHODS_NO_BETA  = ["ce", "pg", "dg"]
METHODS_WITH_BETA = ["entropy_hv", "entropy_lv"]
ALL_METHODS      = METHODS_NO_BETA + METHODS_WITH_BETA

```

```

[3]: # Load data
x_train, y_train, x_val, y_val, x_test, y_test = load_mnist_split(
    val_size=1000, seed=DATA_SEED
)
print(f"Train: {len(x_train):,} | Val: {len(x_val):,} | Test: {len(x_test):,}
      ↪,}")

```

Train: 59,000 | Val: 1,000 | Test: 10,000

1.1 Hyperparameter Sweep (1 seed)

- ce, pg, dg: sweep lr only (5 values)
- entropy_hv, entropy_lv: sweep lr $\times$ beta ($5 \times 5 = 25$ configs each)

Best config selected by **final validation accuracy** (step 10 000).

```

[4]: sweep_results = {m: {} for m in ALL_METHODS}

# Methods without beta
for method in METHODS_NO_BETA:
    for lr in LR_VALUES:
        print(f"[sweep] {method:12s} lr={lr:.0e}")
        sweep_results[method][lr] = train_run(
            method=method, seed=SWEEP_SEED,
            x_train=x_train, y_train=y_train,
            x_val=x_val,      y_val=y_val,
            x_test=x_test,    y_test=y_test,
            lr=lr, **BASE_CFG,
        )

# Methods with beta
for method in METHODS_WITH_BETA:

```

```

for lr, beta in itertools.product(LR_VALUES, BETA_VALUES):
    print(f"[sweep] {method:12s} lr={lr:.0e} beta={beta:.0e}")
    sweep_results[method][(lr, beta)] = train_run(
        method=method, seed=SWEEP_SEED,
        x_train=x_train, y_train=y_train,
        x_val=x_val, y_val=y_val,
        x_test=x_test, y_test=y_test,
        lr=lr, beta=beta, **BASE_CFG,
    )

print("\nSweep complete.")

```

```

[sweep] ce lr=1e-04
[sweep] ce lr=3e-04
[sweep] ce lr=1e-03
[sweep] ce lr=3e-03
[sweep] ce lr=1e-02
[sweep] pg lr=1e-04
[sweep] pg lr=3e-04
[sweep] pg lr=1e-03
[sweep] pg lr=3e-03
[sweep] pg lr=1e-02
[sweep] dg lr=1e-04
[sweep] dg lr=3e-04
[sweep] dg lr=1e-03
[sweep] dg lr=3e-03
[sweep] dg lr=1e-02
[sweep] entropy_hv lr=1e-04 beta=1e-02
[sweep] entropy_hv lr=1e-04 beta=3e-02
[sweep] entropy_hv lr=1e-04 beta=1e-01
[sweep] entropy_hv lr=1e-04 beta=3e-01
[sweep] entropy_hv lr=1e-04 beta=1e+00
[sweep] entropy_hv lr=3e-04 beta=1e-02
[sweep] entropy_hv lr=3e-04 beta=3e-02
[sweep] entropy_hv lr=3e-04 beta=1e-01
[sweep] entropy_hv lr=3e-04 beta=3e-01
[sweep] entropy_hv lr=3e-04 beta=1e+00
[sweep] entropy_hv lr=1e-03 beta=1e-02
[sweep] entropy_hv lr=1e-03 beta=3e-02
[sweep] entropy_hv lr=1e-03 beta=1e-01
[sweep] entropy_hv lr=1e-03 beta=3e-01
[sweep] entropy_hv lr=1e-03 beta=1e+00
[sweep] entropy_hv lr=3e-03 beta=1e-02
[sweep] entropy_hv lr=3e-03 beta=3e-02
[sweep] entropy_hv lr=3e-03 beta=1e-01
[sweep] entropy_hv lr=3e-03 beta=3e-01
[sweep] entropy_hv lr=3e-03 beta=1e+00
[sweep] entropy_hv lr=1e-02 beta=1e-02

```

```

[sweep] entropy_hv      lr=1e-02  beta=3e-02
[sweep] entropy_hv      lr=1e-02  beta=1e-01
[sweep] entropy_hv      lr=1e-02  beta=3e-01
[sweep] entropy_hv      lr=1e-02  beta=1e+00
[sweep] entropy_lv      lr=1e-04  beta=1e-02
[sweep] entropy_lv      lr=1e-04  beta=3e-02
[sweep] entropy_lv      lr=1e-04  beta=1e-01
[sweep] entropy_lv      lr=1e-04  beta=3e-01
[sweep] entropy_lv      lr=1e-04  beta=1e+00
[sweep] entropy_lv      lr=3e-04  beta=1e-02
[sweep] entropy_lv      lr=3e-04  beta=3e-02
[sweep] entropy_lv      lr=3e-04  beta=1e-01
[sweep] entropy_lv      lr=3e-04  beta=3e-01
[sweep] entropy_lv      lr=3e-04  beta=1e+00
[sweep] entropy_lv      lr=1e-03  beta=1e-02
[sweep] entropy_lv      lr=1e-03  beta=3e-02
[sweep] entropy_lv      lr=1e-03  beta=1e-01
[sweep] entropy_lv      lr=1e-03  beta=3e-01
[sweep] entropy_lv      lr=1e-03  beta=1e+00
[sweep] entropy_lv      lr=3e-03  beta=1e-02
[sweep] entropy_lv      lr=3e-03  beta=3e-02
[sweep] entropy_lv      lr=3e-03  beta=1e-01
[sweep] entropy_lv      lr=3e-03  beta=3e-01
[sweep] entropy_lv      lr=3e-03  beta=1e+00
[sweep] entropy_lv      lr=1e-02  beta=1e-02
[sweep] entropy_lv      lr=1e-02  beta=3e-02
[sweep] entropy_lv      lr=1e-02  beta=1e-01
[sweep] entropy_lv      lr=1e-02  beta=3e-01
[sweep] entropy_lv      lr=1e-02  beta=1e+00

```

Sweep complete.

```

[5]: # Select best hparams per method (final val accuracy)
best_hparams = {}

for method in METHODS_NO_BETA:
    best_lr = max(LR_VALUES,
                  key=lambda lr: sweep_results[method][lr]["val_accs"][-1].
    ↪item())
    best_hparams[method] = {"lr": best_lr}

for method in METHODS_WITH_BETA:
    keys      = list(itertools.product(LR_VALUES, BETA_VALUES))
    best_key = max(keys,
                  key=lambda k: sweep_results[method][k]["val_accs"][-1].
    ↪item())
    best_hparams[method] = {"lr": best_key[0], "beta": best_key[1]}

```

```

print("Best hyperparameters (by final val accuracy after sweep):")
print("-" * 50)
for method, hp in best_hparams.items():
    parts = [f"lr={hp['lr']:.0e}"]
    if "beta" in hp:
        parts.append(f"beta={hp['beta']:.0e}")
    print(f"  {method:15s}: {' '.join(parts)}")

```

Best hyperparameters (by final val accuracy after sweep):

```

-----
ce           : lr=3e-03
pg           : lr=1e-03
dg           : lr=1e-03
entropy_hv   : lr=1e-03, beta=3e-01
entropy_lv   : lr=1e-03, beta=3e-01

```

1.2 Final Run (5 seeds, best hparams per method)

```

[9]: final_results = {}
n_seeds = len(FINAL_SEEDS)

def _collect_runs(method, hp):
    """Run *method* with hparams *hp* over FINAL_SEEDS; return aggregated dict.
    ↪ """
    hp_str = ", ".join(f"{k}={v:.0e}" for k, v in hp.items())
    print(f"\n=== {method.upper():12s} | {hp_str} ===")
    runs_val, runs_test, shared_steps = [], [], None
    for seed in FINAL_SEEDS:
        result = train_run(
            method=method, seed=seed,
            x_train=x_train, y_train=y_train,
            x_val=x_val, y_val=y_val,
            x_test=x_test, y_test=y_test,
            **{**BASE_CFG, **hp},
        )
        shared_steps = result["steps"]
        runs_val.append(result["val_accs"])
        runs_test.append(result["test_accs"])
        print(
            f"  seed={seed} "
            f"val={result['val_accs'][-1]:.4f} "
            f"test={result['test_accs'][-1]:.4f}"
        )
    runs_val = torch.stack(runs_val)
    runs_test = torch.stack(runs_test)
    return {

```

```

        "steps"      : shared_steps,
        "val_accs"   : runs_val,
        "test_accs"  : runs_test,
        "val_mean"   : runs_val.mean(0),
        "val_stderr" : runs_val.std(0, unbiased=True) / math.sqrt(n_seeds),
        "test_mean"  : runs_test.mean(0),
        "test_stderr": runs_test.std(0, unbiased=True) / math.sqrt(n_seeds),
    }

# All methods at their best hparams
for method in ALL_METHODS:
    final_results[method] = _collect_runs(method, best_hparams[method])

# CE also at lr=1e-3 (shared lr with all bandit methods, for fair plot)
final_results["ce_1e3"] = _collect_runs("ce", {"lr": 1e-3})

print("\nFinal runs complete.")

```

```

=== CE          | lr=3e-03 ===
seed=0  val=0.9780  test=0.9759
seed=1  val=0.9710  test=0.9676
seed=2  val=0.9790  test=0.9722
seed=3  val=0.9640  test=0.9729
seed=4  val=0.9730  test=0.9738

=== PG          | lr=1e-03 ===
seed=0  val=0.9510  test=0.9590
seed=1  val=0.9510  test=0.9579
seed=2  val=0.9480  test=0.9525
seed=3  val=0.9440  test=0.9497
seed=4  val=0.9440  test=0.9454

=== DG          | lr=1e-03 ===
seed=0  val=0.9680  test=0.9683
seed=1  val=0.9610  test=0.9654
seed=2  val=0.9600  test=0.9678
seed=3  val=0.9580  test=0.9653
seed=4  val=0.9590  test=0.9675

=== ENTROPY_HV   | lr=1e-03, beta=3e-01 ===
seed=0  val=0.9650  test=0.9683
seed=1  val=0.9570  test=0.9651
seed=2  val=0.9610  test=0.9644
seed=3  val=0.9650  test=0.9670
seed=4  val=0.9640  test=0.9680

=== ENTROPY_LV   | lr=1e-03, beta=3e-01 ===

```

```

seed=0  val=0.9700  test=0.9716
seed=1  val=0.9690  test=0.9692
seed=2  val=0.9700  test=0.9715
seed=3  val=0.9620  test=0.9737
seed=4  val=0.9620  test=0.9690

```

```

=== CE          | lr=1e-03 ===
seed=0  val=0.9710  test=0.9720
seed=1  val=0.9740  test=0.9725
seed=2  val=0.9720  test=0.9719
seed=3  val=0.9700  test=0.9735
seed=4  val=0.9650  test=0.9735

```

Final runs complete.

```

[12]: # Plot
# CE is plotted at lr=1e-3 (shared lr) with a dotted line for visual separation.
# Colors: PG=dark orange, DG=dark blue, entropy_hv=red, entropy_lv=purple.
COLORS = {
    "ce_1e3"      : "black",
    "pg"          : "#c45c00", # dark orange
    "dg"          : "#003f7f", # dark blue
    "entropy_hv"  : "#cc0000", # red
    "entropy_lv"  : "#6a0dad", # purple
}
LABELS = {
    "ce_1e3"      : "Cross-Entropy lr=1e-3 (oracle)",
    "pg"          : "Policy Gradient",
    "dg"          : "Delightful PG",
    "entropy_hv"  : "Entropy-HV (exact grad)",
    "entropy_lv"  : "Entropy-LV (score fn)",
}
LINESTYLES = {
    "ce_1e3"      : (0, (5, 3)), # dashed
    "pg"          : "solid",
    "dg"          : "solid",
    "entropy_hv"  : "solid",
    "entropy_lv"  : "solid",
}

PLOT_METHODS = ["ce_1e3", "pg", "dg", "entropy_hv", "entropy_lv"]

fig, ax = plt.subplots(figsize=(8, 5))

for method in PLOT_METHODS:
    res      = final_results[method]
    steps    = res["steps"].numpy()

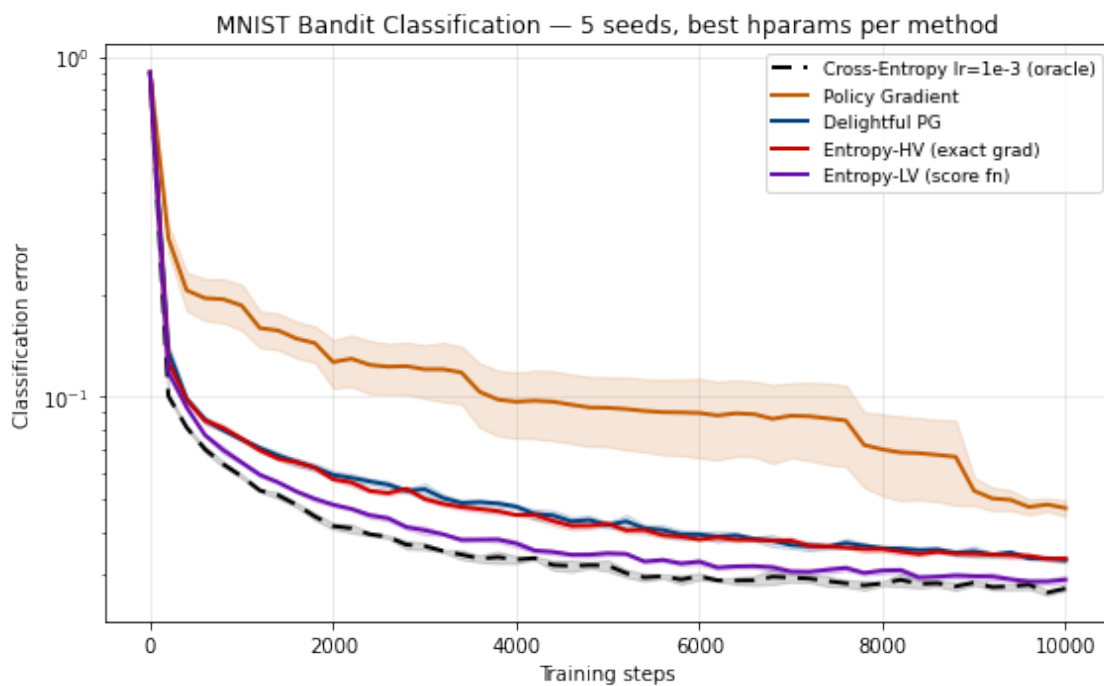
```

```

mean    = 1.0 - res["test_mean"].numpy()
stderr  = res["test_stderr"].numpy()
ax.plot(steps, mean,
        label=LABELS[method],
        color=COLORS[method],
        linestyle=LINESTYLES[method],
        linewidth=2)
ax.fill_between(steps, mean - stderr, mean + stderr,
               color=COLORS[method], alpha=0.15)

ax.set_yscale("log")
ax.set_xlabel("Training steps")
ax.set_ylabel("Classification error")
ax.set_title(f"MNIST Bandit Classification - {n_seeds} seeds, best hparams per_
↳method")
ax.grid(alpha=0.3)
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()

```



```

[13]: # Summary
print("=" * 62)
print("BEST HYPERPARAMETERS (selected by final val accuracy, sweep)")
print("=" * 62)

```

```

for method, hp in best_hparams.items():
    parts = [f"lr={hp['lr']:.0e}"]
    if "beta" in hp:
        parts.append(f"beta={hp['beta']:.0e}")
    print(f" {method:15s}: {' '.join(parts)}")

print()
print("=" * 55)
print(f"FINAL TEST RESULTS (mean ± stderr over {n_seeds} seeds)")
print("=" * 55)
for method in ALL_METHODS:
    res = final_results[method]
    t_err = 1.0 - res["test_mean"][-1].item()
    t_se = res["test_stderr"][-1].item()
    print(f" {method:15s}: test_err={t_err:.4f}±{t_se:.4f}")

# CE at lr=1e-3 (the curve shown in the plot)
res = final_results["ce_1e3"]
t_err = 1.0 - res["test_mean"][-1].item()
t_se = res["test_stderr"][-1].item()
print(f" {'ce (lr=1e-3)':15s}: test_err={t_err:.4f}±{t_se:.4f} (plotted)")

```

```

=====
BEST HYPERPARAMETERS (selected by final val accuracy, sweep)
=====

```

```

ce           : lr=3e-03
pg           : lr=1e-03
dg           : lr=1e-03
entropy_hv   : lr=1e-03, beta=3e-01
entropy_lv   : lr=1e-03, beta=3e-01

```

```

=====
FINAL TEST RESULTS (mean ± stderr over 5 seeds)
=====

```

```

ce           : test_err=0.0275±0.0014
pg           : test_err=0.0471±0.0025
dg           : test_err=0.0331±0.0006
entropy_hv   : test_err=0.0334±0.0008
entropy_lv   : test_err=0.0290±0.0009
ce (lr=1e-3) : test_err=0.0273±0.0003 (plotted)

```

```
[14]: final_results
```

```

[14]: {'ce': {'steps': tensor([  0,  200,  400,  600,  800, 1000, 1200, 1400,
1600, 1800,
          2000, 2200, 2400, 2600, 2800, 3000, 3200, 3400, 3600, 3800,
          4000, 4200, 4400, 4600, 4800, 5000, 5200, 5400, 5600, 5800,
          6000, 6200, 6400, 6600, 6800, 7000, 7200, 7400, 7600, 7800,

```

```

8000, 8200, 8400, 8600, 8800, 9000, 9200, 9400, 9600, 9800,
10000]),
'val_accs': tensor([[0.0800, 0.9160, 0.9350, 0.9480, 0.9520, 0.9460, 0.9580,
0.9520, 0.9520,
0.9600, 0.9560, 0.9470, 0.9620, 0.9670, 0.9640, 0.9630, 0.9670,
0.9640,
0.9620, 0.9630, 0.9680, 0.9680, 0.9670, 0.9690, 0.9670, 0.9680,
0.9680,
0.9700, 0.9710, 0.9680, 0.9670, 0.9740, 0.9670, 0.9750, 0.9660,
0.9650,
0.9670, 0.9690, 0.9690, 0.9660, 0.9680, 0.9690, 0.9670, 0.9700,
0.9680,
0.9650, 0.9710, 0.9730, 0.9740, 0.9690, 0.9780],
[0.1140, 0.9130, 0.9290, 0.9490, 0.9510, 0.9500, 0.9480, 0.9550,
0.9510,
0.9660, 0.9580, 0.9610, 0.9530, 0.9630, 0.9670, 0.9620, 0.9600,
0.9630,
0.9640, 0.9610, 0.9670, 0.9670, 0.9660, 0.9580, 0.9630, 0.9730,
0.9740,
0.9710, 0.9700, 0.9710, 0.9730, 0.9700, 0.9740, 0.9680, 0.9640,
0.9670,
0.9680, 0.9720, 0.9700, 0.9740, 0.9720, 0.9700, 0.9640, 0.9630,
0.9720,
0.9670, 0.9750, 0.9740, 0.9710, 0.9670, 0.9710],
[0.0900, 0.9100, 0.9420, 0.9430, 0.9500, 0.9550, 0.9600, 0.9560,
0.9550,
0.9540, 0.9600, 0.9630, 0.9660, 0.9630, 0.9670, 0.9760, 0.9710,
0.9680,
0.9690, 0.9670, 0.9600, 0.9760, 0.9680, 0.9690, 0.9780, 0.9700,
0.9740,
0.9730, 0.9730, 0.9670, 0.9760, 0.9790, 0.9760, 0.9690, 0.9750,
0.9740,
0.9780, 0.9730, 0.9740, 0.9810, 0.9740, 0.9720, 0.9740, 0.9770,
0.9730,
0.9780, 0.9800, 0.9730, 0.9770, 0.9700, 0.9790],
[0.1200, 0.9100, 0.9370, 0.9420, 0.9470, 0.9530, 0.9540, 0.9540,
0.9520,
0.9580, 0.9610, 0.9620, 0.9700, 0.9640, 0.9710, 0.9620, 0.9650,
0.9560,
0.9610, 0.9690, 0.9640, 0.9650, 0.9680, 0.9690, 0.9670, 0.9720,
0.9610,
0.9670, 0.9670, 0.9710, 0.9670, 0.9730, 0.9740, 0.9700, 0.9660,
0.9720,
0.9720, 0.9660, 0.9660, 0.9670, 0.9670, 0.9690, 0.9680, 0.9680,
0.9650,
0.9650, 0.9670, 0.9650, 0.9560, 0.9660, 0.9640],
[0.0750, 0.9130, 0.9280, 0.9440, 0.9510, 0.9550, 0.9390, 0.9410,

```

0.9640,
 0.9570, 0.9620, 0.9590, 0.9530, 0.9580, 0.9620, 0.9740, 0.9690,
 0.9670,
 0.9670, 0.9770, 0.9770, 0.9690, 0.9700, 0.9600, 0.9680, 0.9620,
 0.9740,
 0.9710, 0.9670, 0.9690, 0.9650, 0.9640, 0.9740, 0.9690, 0.9730,
 0.9710,
 0.9660, 0.9700, 0.9710, 0.9680, 0.9660, 0.9730, 0.9510, 0.9760,
 0.9660,
 0.9710, 0.9640, 0.9690, 0.9620, 0.9600, 0.9730]]),
 'test_accs': tensor([[0.0926, 0.9267, 0.9405, 0.9509, 0.9587, 0.9533, 0.9621,
 0.9634, 0.9655,
 0.9666, 0.9670, 0.9637, 0.9674, 0.9695, 0.9704, 0.9713, 0.9678,
 0.9717,
 0.9717, 0.9689, 0.9713, 0.9737, 0.9692, 0.9731, 0.9741, 0.9734,
 0.9746,
 0.9762, 0.9742, 0.9728, 0.9762, 0.9754, 0.9715, 0.9734, 0.9758,
 0.9695,
 0.9737, 0.9705, 0.9764, 0.9740, 0.9742, 0.9748, 0.9744, 0.9728,
 0.9745,
 0.9708, 0.9760, 0.9716, 0.9734, 0.9728, 0.9759],
 [0.1375, 0.9148, 0.9286, 0.9444, 0.9461, 0.9551, 0.9585, 0.9584,
 0.9622,
 0.9565, 0.9654, 0.9664, 0.9613, 0.9651, 0.9614, 0.9660, 0.9687,
 0.9679,
 0.9672, 0.9692, 0.9696, 0.9683, 0.9728, 0.9684, 0.9674, 0.9713,
 0.9729,
 0.9686, 0.9683, 0.9733, 0.9735, 0.9706, 0.9713, 0.9710, 0.9679,
 0.9697,
 0.9728, 0.9700, 0.9715, 0.9725, 0.9731, 0.9715, 0.9695, 0.9720,
 0.9740,
 0.9721, 0.9741, 0.9733, 0.9728, 0.9700, 0.9676],
 [0.0990, 0.9176, 0.9462, 0.9514, 0.9549, 0.9614, 0.9645, 0.9615,
 0.9617,
 0.9612, 0.9624, 0.9664, 0.9669, 0.9684, 0.9724, 0.9701, 0.9684,
 0.9725,
 0.9703, 0.9691, 0.9642, 0.9686, 0.9724, 0.9701, 0.9676, 0.9689,
 0.9734,
 0.9713, 0.9737, 0.9697, 0.9704, 0.9728, 0.9737, 0.9690, 0.9728,
 0.9721,
 0.9757, 0.9697, 0.9714, 0.9709, 0.9749, 0.9740, 0.9713, 0.9734,
 0.9729,
 0.9714, 0.9738, 0.9710, 0.9732, 0.9716, 0.9722],
 [0.1135, 0.9153, 0.9365, 0.9467, 0.9560, 0.9557, 0.9600, 0.9603,
 0.9576,
 0.9646, 0.9658, 0.9628, 0.9680, 0.9662, 0.9674, 0.9660, 0.9691,
 0.9700,

```

0.9656, 0.9716, 0.9669, 0.9635, 0.9720, 0.9716, 0.9729, 0.9720,
0.9725,
0.9737, 0.9740, 0.9703, 0.9723, 0.9719, 0.9722, 0.9714, 0.9668,
0.9738,
0.9710, 0.9729, 0.9744, 0.9733, 0.9730, 0.9732, 0.9744, 0.9730,
0.9719,
0.9723, 0.9724, 0.9740, 0.9708, 0.9722, 0.9729],
[0.0634, 0.9095, 0.9360, 0.9510, 0.9537, 0.9568, 0.9580, 0.9576,
0.9593,
0.9629, 0.9663, 0.9681, 0.9655, 0.9686, 0.9671, 0.9689, 0.9689,
0.9713,
0.9676, 0.9691, 0.9706, 0.9698, 0.9716, 0.9682, 0.9698, 0.9668,
0.9694,
0.9725, 0.9688, 0.9720, 0.9723, 0.9692, 0.9721, 0.9721, 0.9691,
0.9719,
0.9711, 0.9743, 0.9715, 0.9694, 0.9728, 0.9710, 0.9674, 0.9731,
0.9748,
0.9714, 0.9737, 0.9700, 0.9692, 0.9699, 0.9738]]),
'val_mean': tensor([0.0958, 0.9124, 0.9342, 0.9452, 0.9502, 0.9518, 0.9518,
0.9516, 0.9548,
0.9590, 0.9594, 0.9584, 0.9608, 0.9630, 0.9662, 0.9674, 0.9664,
0.9636,
0.9646, 0.9674, 0.9672, 0.9690, 0.9678, 0.9650, 0.9686, 0.9690,
0.9702,
0.9704, 0.9696, 0.9692, 0.9696, 0.9720, 0.9730, 0.9702, 0.9688,
0.9698,
0.9702, 0.9700, 0.9700, 0.9712, 0.9694, 0.9706, 0.9648, 0.9708,
0.9688,
0.9692, 0.9714, 0.9708, 0.9680, 0.9664, 0.9730])),
'val_stderr': tensor([0.0090, 0.0011, 0.0026, 0.0014, 0.0009, 0.0017, 0.0038,
0.0027, 0.0024,
0.0020, 0.0011, 0.0029, 0.0034, 0.0014, 0.0015, 0.0031, 0.0019,
0.0021,
0.0015, 0.0028, 0.0028, 0.0019, 0.0007, 0.0025, 0.0025, 0.0019,
0.0026,
0.0010, 0.0012, 0.0008, 0.0021, 0.0025, 0.0015, 0.0012, 0.0022,
0.0017,
0.0022, 0.0012, 0.0013, 0.0028, 0.0015, 0.0008, 0.0038, 0.0026,
0.0016,
0.0025, 0.0028, 0.0017, 0.0039, 0.0017, 0.0027])),
'test_mean': tensor([0.1012, 0.9168, 0.9376, 0.9489, 0.9539, 0.9565, 0.9606,
0.9602, 0.9613,
0.9624, 0.9654, 0.9655, 0.9658, 0.9676, 0.9677, 0.9685, 0.9686,
0.9707,
0.9685, 0.9696, 0.9685, 0.9688, 0.9716, 0.9703, 0.9704, 0.9705,
0.9726,
0.9725, 0.9718, 0.9716, 0.9729, 0.9720, 0.9722, 0.9714, 0.9705,

```

```

0.9714,
    0.9729, 0.9715, 0.9730, 0.9720, 0.9736, 0.9729, 0.9714, 0.9729,
0.9736,
    0.9716, 0.9740, 0.9720, 0.9719, 0.9713, 0.9725]],
    'test_stderr': tensor([0.0122, 0.0028, 0.0029, 0.0014, 0.0021, 0.0014, 0.0012,
0.0010, 0.0013,
    0.0017, 0.0008, 0.0010, 0.0012, 0.0008, 0.0019, 0.0011, 0.0002,
0.0008,
    0.0011, 0.0005, 0.0013, 0.0016, 0.0006, 0.0009, 0.0014, 0.0012,
0.0009,
    0.0013, 0.0013, 0.0007, 0.0010, 0.0010, 0.0004, 0.0007, 0.0017,
0.0008,
    0.0009, 0.0009, 0.0010, 0.0008, 0.0004, 0.0007, 0.0014, 0.0002,
0.0005,
    0.0003, 0.0006, 0.0007, 0.0008, 0.0006, 0.0014]))},
    'pg': {'steps': tensor([  0,  200,  400,  600,  800, 1000, 1200, 1400,
1600, 1800,
    2000, 2200, 2400, 2600, 2800, 3000, 3200, 3400, 3600, 3800,
    4000, 4200, 4400, 4600, 4800, 5000, 5200, 5400, 5600, 5800,
    6000, 6200, 6400, 6600, 6800, 7000, 7200, 7400, 7600, 7800,
    8000, 8200, 8400, 8600, 8800, 9000, 9200, 9400, 9600, 9800,
    10000])),
    'val_accs': tensor([[0.0800, 0.7740, 0.8780, 0.8930, 0.9090, 0.9050, 0.9070,
0.9130, 0.9080,
    0.9120, 0.9220, 0.9190, 0.9360, 0.9230, 0.9330, 0.9290, 0.9400,
0.9310,
    0.9320, 0.9360, 0.9320, 0.9320, 0.9390, 0.9360, 0.9400, 0.9410,
0.9400,
    0.9460, 0.9430, 0.9460, 0.9440, 0.9420, 0.9440, 0.9500, 0.9450,
0.9510,
    0.9500, 0.9470, 0.9530, 0.9480, 0.9530, 0.9400, 0.9490, 0.9500,
0.9490,
    0.9470, 0.9510, 0.9550, 0.9480, 0.9480, 0.9510],
[0.1140, 0.7030, 0.7240, 0.7330, 0.7350, 0.7410, 0.8090, 0.8220,
0.8180,
    0.8430, 0.8990, 0.9160, 0.9130, 0.9200, 0.9260, 0.9220, 0.9270,
0.9220,
    0.9290, 0.9250, 0.9260, 0.9310, 0.9340, 0.9260, 0.9270, 0.9440,
0.9380,
    0.9450, 0.9350, 0.9490, 0.9380, 0.9440, 0.9360, 0.9340, 0.9440,
0.9330,
    0.9510, 0.9470, 0.9420, 0.9450, 0.9400, 0.9550, 0.9510, 0.9520,
0.9520,
    0.9450, 0.9450, 0.9470, 0.9480, 0.9390, 0.9510],
[0.0900, 0.6580, 0.8110, 0.8150, 0.8210, 0.8270, 0.8340, 0.8270,
0.8330,
    0.8410, 0.8340, 0.8390, 0.8380, 0.8520, 0.8450, 0.8510, 0.8370,

```

0.8510,
 0.9300, 0.9160, 0.9290, 0.9300, 0.9300, 0.9260, 0.9340, 0.9370, 0.9380,
 0.9450, 0.9270, 0.9420, 0.9290, 0.9380, 0.9380, 0.9410, 0.9370, 0.9420,
 0.9490, 0.9390, 0.9430, 0.9410, 0.9410, 0.9390, 0.9460, 0.9430, 0.9500,
 0.9430, 0.9460, 0.9500, 0.9530, 0.9520, 0.9480],
 [0.1200, 0.7780, 0.8100, 0.8230, 0.8320, 0.8300, 0.8390, 0.8260,
 0.8340, 0.8360, 0.8420, 0.8340, 0.8380, 0.8480, 0.8480, 0.8450, 0.8360,
 0.8470, 0.8390, 0.8350, 0.8590, 0.8530, 0.8560, 0.8540, 0.8490, 0.8620,
 0.8550, 0.8570, 0.8590, 0.8600, 0.8610, 0.8640, 0.8610, 0.8560, 0.8600,
 0.8600, 0.8650, 0.8650, 0.9240, 0.9360, 0.9420, 0.9400, 0.9500,
 0.9430, 0.9430, 0.9470, 0.9520, 0.9580, 0.9490, 0.9440],
 [0.0750, 0.6330, 0.7330, 0.7420, 0.7480, 0.7500, 0.8140, 0.8220,
 0.8250, 0.8170, 0.8330, 0.8230, 0.8350, 0.8390, 0.8360, 0.8390, 0.8420,
 0.8350, 0.8380, 0.8480, 0.8480, 0.8420, 0.8460, 0.8410, 0.8470, 0.8450,
 0.8480, 0.8460, 0.8560, 0.8480, 0.8430, 0.8530, 0.8530, 0.8540, 0.8550,
 0.8560, 0.8520, 0.8500, 0.8520, 0.8540, 0.8490, 0.8500, 0.8600, 0.8570,
 0.8580, 0.9230, 0.9430, 0.9380, 0.9470, 0.9450, 0.9440]]),
 'test_accs': tensor([[0.0926, 0.7817, 0.8823, 0.8990, 0.9016, 0.9101, 0.9176,
 0.9198, 0.9243,
 0.9260, 0.9289, 0.9269, 0.9359, 0.9294, 0.9331, 0.9320, 0.9392,
 0.9391, 0.9398, 0.9418, 0.9383, 0.9400, 0.9432, 0.9447, 0.9454, 0.9432,
 0.9466, 0.9462, 0.9461, 0.9461, 0.9486, 0.9495, 0.9471, 0.9498, 0.9526,
 0.9532, 0.9507, 0.9503, 0.9547, 0.9533, 0.9537, 0.9522, 0.9540, 0.9552,
 0.9530, 0.9548, 0.9567, 0.9537, 0.9568, 0.9581, 0.9590],
 [0.1375, 0.7087, 0.7324, 0.7438, 0.7389, 0.7496, 0.8156, 0.8271,
 0.8293, 0.8463, 0.9180, 0.9226, 0.9237, 0.9269, 0.9250, 0.9320, 0.9350,
 0.9358, 0.9358, 0.9340, 0.9393, 0.9379, 0.9414, 0.9354, 0.9406, 0.9419,
 0.9426,

0.9487, 0.9463, 0.9495, 0.9503, 0.9522, 0.9446, 0.9478, 0.9493,
 0.9395,
 0.9518, 0.9526, 0.9506, 0.9496, 0.9497, 0.9542, 0.9511, 0.9538,
 0.9538,
 0.9525, 0.9559, 0.9561, 0.9580, 0.9504, 0.9579],
 [0.0990, 0.6526, 0.8058, 0.8159, 0.8123, 0.8235, 0.8261, 0.8151,
 0.8308,
 0.8367, 0.8353, 0.8304, 0.8364, 0.8398, 0.8372, 0.8424, 0.8308,
 0.8399,
 0.9216, 0.9342, 0.9360, 0.9379, 0.9346, 0.9419, 0.9404, 0.9415,
 0.9433,
 0.9394, 0.9475, 0.9457, 0.9468, 0.9457, 0.9502, 0.9446, 0.9494,
 0.9498,
 0.9450, 0.9511, 0.9508, 0.9513, 0.9495, 0.9518, 0.9506, 0.9512,
 0.9528,
 0.9529, 0.9522, 0.9546, 0.9511, 0.9547, 0.9525],
 [0.1135, 0.7563, 0.8074, 0.8098, 0.8228, 0.8267, 0.8330, 0.8287,
 0.8377,
 0.8411, 0.8434, 0.8367, 0.8421, 0.8465, 0.8440, 0.8441, 0.8440,
 0.8508,
 0.8402, 0.8496, 0.8515, 0.8522, 0.8504, 0.8510, 0.8560, 0.8573,
 0.8548,
 0.8586, 0.8580, 0.8582, 0.8583, 0.8580, 0.8549, 0.8554, 0.8604,
 0.8598,
 0.8593, 0.8601, 0.8615, 0.9267, 0.9422, 0.9434, 0.9448, 0.9424,
 0.9482,
 0.9477, 0.9489, 0.9495, 0.9529, 0.9504, 0.9497],
 [0.0634, 0.6381, 0.7434, 0.7561, 0.7560, 0.7616, 0.8114, 0.8258,
 0.8363,
 0.8304, 0.8415, 0.8362, 0.8411, 0.8440, 0.8452, 0.8468, 0.8482,
 0.8444,
 0.8452, 0.8495, 0.8511, 0.8447, 0.8465, 0.8528, 0.8523, 0.8519,
 0.8524,
 0.8526, 0.8514, 0.8506, 0.8478, 0.8539, 0.8560, 0.8579, 0.8576,
 0.8577,
 0.8549, 0.8538, 0.8563, 0.8581, 0.8547, 0.8554, 0.8580, 0.8599,
 0.8590,
 0.9273, 0.9350, 0.9372, 0.9440, 0.9453, 0.9454]]),
 'val_mean': tensor([0.0958, 0.7092, 0.7912, 0.8012, 0.8090, 0.8106, 0.8406,
 0.8420, 0.8436,
 0.8498, 0.8660, 0.8662, 0.8720, 0.8764, 0.8776, 0.8772, 0.8764,
 0.8772,
 0.8908, 0.8946, 0.8990, 0.8976, 0.9002, 0.8982, 0.9000, 0.9060,
 0.9022,
 0.9042, 0.9070, 0.9064, 0.9048, 0.9082, 0.9070, 0.9062, 0.9092,
 0.9090,
 0.9104, 0.9104, 0.9106, 0.9224, 0.9234, 0.9266, 0.9286, 0.9318,

```

0.9302,
    0.9402, 0.9464, 0.9484, 0.9508, 0.9466, 0.9476]),
    'val_stderr': tensor([0.0090, 0.0295, 0.0284, 0.0294, 0.0315, 0.0301, 0.0176,
0.0178, 0.0164,
    0.0162, 0.0186, 0.0211, 0.0217, 0.0185, 0.0213, 0.0198, 0.0234,
0.0203,
    0.0215, 0.0218, 0.0187, 0.0205, 0.0203, 0.0209, 0.0213, 0.0216,
0.0208,
    0.0218, 0.0203, 0.0217, 0.0218, 0.0204, 0.0205, 0.0211, 0.0211,
0.0210,
    0.0223, 0.0217, 0.0215, 0.0176, 0.0188, 0.0193, 0.0173, 0.0187,
0.0181,
    0.0044, 0.0013, 0.0029, 0.0021, 0.0022, 0.0016])),
    'test_mean': tensor([0.1012, 0.7075, 0.7943, 0.8049, 0.8063, 0.8143, 0.8407,
0.8433, 0.8517,
    0.8561, 0.8734, 0.8706, 0.8758, 0.8773, 0.8769, 0.8795, 0.8794,
0.8820,
    0.8965, 0.9018, 0.9032, 0.9025, 0.9032, 0.9052, 0.9069, 0.9072,
0.9079,
    0.9091, 0.9099, 0.9100, 0.9104, 0.9119, 0.9106, 0.9111, 0.9139,
0.9120,
    0.9123, 0.9136, 0.9148, 0.9278, 0.9300, 0.9314, 0.9317, 0.9325,
0.9334,
    0.9470, 0.9497, 0.9502, 0.9526, 0.9518, 0.9529])),
    'test_stderr': tensor([0.0122, 0.0280, 0.0269, 0.0275, 0.0287, 0.0286, 0.0196,
0.0193, 0.0182,
    0.0177, 0.0205, 0.0222, 0.0221, 0.0208, 0.0214, 0.0215, 0.0237,
0.0227,
    0.0222, 0.0214, 0.0212, 0.0221, 0.0224, 0.0218, 0.0216, 0.0215,
0.0222,
    0.0219, 0.0225, 0.0227, 0.0235, 0.0229, 0.0225, 0.0222, 0.0224,
0.0219,
    0.0226, 0.0231, 0.0228, 0.0181, 0.0189, 0.0191, 0.0185, 0.0183,
0.0186,
    0.0051, 0.0039, 0.0034, 0.0025, 0.0022, 0.0025]))},
    'dg': {'steps': tensor([ 0, 200, 400, 600, 800, 1000, 1200, 1400,
1600, 1800,
    2000, 2200, 2400, 2600, 2800, 3000, 3200, 3400, 3600, 3800,
4000, 4200, 4400, 4600, 4800, 5000, 5200, 5400, 5600, 5800,
6000, 6200, 6400, 6600, 6800, 7000, 7200, 7400, 7600, 7800,
8000, 8200, 8400, 8600, 8800, 9000, 9200, 9400, 9600, 9800,
10000])},
    'val_accs': tensor([[0.0800, 0.8720, 0.8900, 0.9130, 0.9220, 0.9190, 0.9290,
0.9260, 0.9300,
    0.9420, 0.9420, 0.9370, 0.9470, 0.9440, 0.9450, 0.9430, 0.9460,
0.9490,
    0.9480, 0.9500, 0.9450, 0.9500, 0.9510, 0.9530, 0.9550, 0.9480,

```

0.9510,
 0.9510,
 0.9590,
 0.9310,
 0.9360,
 0.9510,
 0.9600,
 0.9580,
 0.9340,
 0.9450,
 0.9610,
 0.9600,
 0.9690,
 0.9290,
 0.9420,
 0.9490,
 0.9560,
 0.9530,
 0.9390,
 0.9500,
 0.9490,
 0.9540,
 0.9500, 0.9540, 0.9530, 0.9460, 0.9590, 0.9510, 0.9500, 0.9580,
 0.9620, 0.9560, 0.9560, 0.9520, 0.9520, 0.9580, 0.9460, 0.9560,
 0.9610, 0.9620, 0.9590, 0.9670, 0.9570, 0.9680],
 [0.1140, 0.8750, 0.8960, 0.9040, 0.9210, 0.9150, 0.9170, 0.9320,
 0.9270, 0.9330, 0.9390, 0.9260, 0.9380, 0.9440, 0.9390, 0.9380,
 0.9480, 0.9410, 0.9460, 0.9440, 0.9430, 0.9470, 0.9520, 0.9580,
 0.9630, 0.9480, 0.9530, 0.9530, 0.9490, 0.9540, 0.9560, 0.9570,
 0.9610, 0.9620, 0.9570, 0.9550, 0.9580, 0.9590, 0.9590, 0.9590,
 0.9540, 0.9640, 0.9580, 0.9600, 0.9630, 0.9610],
 [0.0900, 0.8730, 0.8920, 0.9120, 0.9180, 0.9250, 0.9240, 0.9340,
 0.9350, 0.9360, 0.9420, 0.9430, 0.9440, 0.9490, 0.9500, 0.9540,
 0.9490, 0.9540, 0.9520, 0.9530, 0.9540, 0.9560, 0.9520, 0.9570,
 0.9570, 0.9560, 0.9550, 0.9640, 0.9590, 0.9570, 0.9610, 0.9550,
 0.9560, 0.9600, 0.9570, 0.9590, 0.9580, 0.9650, 0.9680, 0.9610,
 0.9600, 0.9630, 0.9640, 0.9560, 0.9610, 0.9600],
 [0.1200, 0.8350, 0.8890, 0.9030, 0.9180, 0.9220, 0.9220, 0.9190,
 0.9260, 0.9370, 0.9400, 0.9400, 0.9480, 0.9460, 0.9440, 0.9440,
 0.9530, 0.9470, 0.9510, 0.9510, 0.9490, 0.9560, 0.9560, 0.9550,
 0.9570, 0.9500, 0.9530, 0.9550, 0.9520, 0.9520, 0.9490, 0.9550,
 0.9480, 0.9560, 0.9530, 0.9570, 0.9540, 0.9550, 0.9580, 0.9600,
 0.9580, 0.9610, 0.9580, 0.9610, 0.9550, 0.9580],
 [0.0750, 0.8550, 0.9030, 0.9080, 0.9180, 0.9210, 0.9260, 0.9270,
 0.9370, 0.9330, 0.9380, 0.9450, 0.9390, 0.9430, 0.9410, 0.9420,
 0.9490, 0.9530, 0.9440, 0.9530, 0.9530, 0.9510, 0.9530, 0.9520,
 0.9500, 0.9530, 0.9520, 0.9520, 0.9500, 0.9550, 0.9580, 0.9510,
 0.9540,

0.9490, 0.9580, 0.9560, 0.9570, 0.9620, 0.9610, 0.9580, 0.9590,
 0.9500,
 0.9530, 0.9560, 0.9590, 0.9560, 0.9610, 0.9590]]),
 'test_accs': tensor([[0.0926, 0.8725, 0.9006, 0.9160, 0.9221, 0.9255, 0.9323,
 0.9328, 0.9394,
 0.9421, 0.9437, 0.9468, 0.9446, 0.9466, 0.9486, 0.9524, 0.9535,
 0.9542,
 0.9530, 0.9535, 0.9522, 0.9585, 0.9571, 0.9605, 0.9620, 0.9606,
 0.9627,
 0.9608, 0.9638, 0.9651, 0.9630, 0.9651, 0.9640, 0.9628, 0.9672,
 0.9666,
 0.9672, 0.9642, 0.9660, 0.9658, 0.9649, 0.9657, 0.9657, 0.9654,
 0.9679,
 0.9682, 0.9654, 0.9676, 0.9670, 0.9681, 0.9683],
 [0.1375, 0.8748, 0.9073, 0.9127, 0.9199, 0.9252, 0.9258, 0.9328,
 0.9356,
 0.9354, 0.9400, 0.9419, 0.9406, 0.9441, 0.9441, 0.9410, 0.9473,
 0.9504,
 0.9508, 0.9481, 0.9531, 0.9534, 0.9488, 0.9550, 0.9545, 0.9575,
 0.9556,
 0.9605, 0.9559, 0.9588, 0.9600, 0.9565, 0.9610, 0.9597, 0.9606,
 0.9629,
 0.9640, 0.9620, 0.9616, 0.9628, 0.9645, 0.9656, 0.9664, 0.9657,
 0.9653,
 0.9664, 0.9669, 0.9643, 0.9657, 0.9656, 0.9654],
 [0.0990, 0.8756, 0.9002, 0.9180, 0.9229, 0.9288, 0.9326, 0.9393,
 0.9369,
 0.9413, 0.9453, 0.9451, 0.9450, 0.9476, 0.9486, 0.9508, 0.9521,
 0.9500,
 0.9505, 0.9525, 0.9546, 0.9584, 0.9584, 0.9599, 0.9542, 0.9593,
 0.9597,
 0.9602, 0.9613, 0.9597, 0.9623, 0.9631, 0.9582, 0.9625, 0.9636,
 0.9629,
 0.9646, 0.9642, 0.9628, 0.9645, 0.9643, 0.9641, 0.9640, 0.9647,
 0.9629,
 0.9637, 0.9673, 0.9641, 0.9663, 0.9670, 0.9678],
 [0.1135, 0.8416, 0.8961, 0.9075, 0.9176, 0.9236, 0.9268, 0.9288,
 0.9311,
 0.9328, 0.9353, 0.9371, 0.9408, 0.9398, 0.9465, 0.9459, 0.9465,
 0.9491,
 0.9515, 0.9504, 0.9504, 0.9514, 0.9551, 0.9533, 0.9548, 0.9570,
 0.9560,
 0.9577, 0.9572, 0.9604, 0.9585, 0.9613, 0.9596, 0.9627, 0.9616,
 0.9622,
 0.9612, 0.9650, 0.9649, 0.9619, 0.9642, 0.9610, 0.9643, 0.9643,
 0.9652,
 0.9655, 0.9644, 0.9650, 0.9659, 0.9663, 0.9653],

```
[0.0634, 0.8539, 0.9026, 0.9176, 0.9183, 0.9216, 0.9288, 0.9303,
0.9352,
    0.9383, 0.9409, 0.9401, 0.9469, 0.9459, 0.9474, 0.9425, 0.9484,
0.9524,
    0.9489, 0.9526, 0.9519, 0.9521, 0.9564, 0.9560, 0.9573, 0.9556,
0.9508,
    0.9554, 0.9588, 0.9588, 0.9592, 0.9599, 0.9615, 0.9613, 0.9580,
0.9621,
    0.9608, 0.9631, 0.9589, 0.9625, 0.9626, 0.9648, 0.9625, 0.9620,
0.9649,
    0.9605, 0.9644, 0.9647, 0.9675, 0.9654, 0.9675]]),
'val_mean': tensor([0.0958, 0.8620, 0.8940, 0.9080, 0.9194, 0.9204, 0.9236,
0.9276, 0.9326,
    0.9334, 0.9362, 0.9392, 0.9402, 0.9426, 0.9454, 0.9434, 0.9448,
0.9444,
    0.9494, 0.9490, 0.9476, 0.9502, 0.9500, 0.9526, 0.9536, 0.9540,
0.9522,
    0.9554, 0.9522, 0.9532, 0.9540, 0.9538, 0.9538, 0.9548, 0.9552,
0.9562,
    0.9552, 0.9584, 0.9558, 0.9560, 0.9568, 0.9596, 0.9578, 0.9590,
0.9578,
    0.9572, 0.9612, 0.9596, 0.9600, 0.9594, 0.9612])),
'val_stderr': tensor([0.0090, 0.0076, 0.0025, 0.0020, 0.0009, 0.0017, 0.0020,
0.0026, 0.0018,
    0.0030, 0.0017, 0.0009, 0.0037, 0.0018, 0.0010, 0.0019, 0.0027,
0.0025,
    0.0009, 0.0023, 0.0016, 0.0017, 0.0019, 0.0017, 0.0008, 0.0018,
0.0022,
    0.0025, 0.0014, 0.0005, 0.0029, 0.0022, 0.0011, 0.0023, 0.0012,
0.0017,
    0.0029, 0.0012, 0.0007, 0.0012, 0.0017, 0.0017, 0.0035, 0.0008,
0.0032,
    0.0016, 0.0014, 0.0011, 0.0020, 0.0015, 0.0018])),
'test_mean': tensor([0.1012, 0.8637, 0.9014, 0.9144, 0.9202, 0.9249, 0.9293,
0.9328, 0.9356,
    0.9380, 0.9410, 0.9422, 0.9436, 0.9448, 0.9470, 0.9465, 0.9496,
0.9512,
    0.9509, 0.9514, 0.9524, 0.9548, 0.9552, 0.9569, 0.9566, 0.9580,
0.9570,
    0.9589, 0.9594, 0.9606, 0.9606, 0.9612, 0.9609, 0.9618, 0.9622,
0.9633,
    0.9636, 0.9637, 0.9628, 0.9635, 0.9641, 0.9642, 0.9646, 0.9644,
0.9652,
    0.9649, 0.9657, 0.9651, 0.9665, 0.9665, 0.9669])),
'test_stderr': tensor([0.0122, 0.0068, 0.0018, 0.0020, 0.0010, 0.0012, 0.0014,
0.0018, 0.0014,
    0.0018, 0.0017, 0.0017, 0.0012, 0.0014, 0.0008, 0.0022, 0.0014,
```

```

0.0009,
    0.0007, 0.0010, 0.0007, 0.0015, 0.0017, 0.0014, 0.0015, 0.0009,
0.0020,
    0.0010, 0.0014, 0.0012, 0.0009, 0.0015, 0.0010, 0.0006, 0.0015,
0.0008,
    0.0012, 0.0005, 0.0013, 0.0007, 0.0004, 0.0009, 0.0007, 0.0007,
0.0008,
    0.0013, 0.0006, 0.0006, 0.0003, 0.0005, 0.0006]]},
'entropy_hv': {'steps': tensor([  0,  200,  400,  600,  800, 1000,
1200, 1400, 1600, 1800,
    2000, 2200, 2400, 2600, 2800, 3000, 3200, 3400, 3600, 3800,
    4000, 4200, 4400, 4600, 4800, 5000, 5200, 5400, 5600, 5800,
    6000, 6200, 6400, 6600, 6800, 7000, 7200, 7400, 7600, 7800,
    8000, 8200, 8400, 8600, 8800, 9000, 9200, 9400, 9600, 9800,
    10000])),
'val_accs': tensor([[0.0800, 0.8760, 0.9000, 0.9100, 0.9230, 0.9230, 0.9320,
0.9260, 0.9290,
    0.9320, 0.9320, 0.9370, 0.9450, 0.9480, 0.9430, 0.9440, 0.9460,
0.9450,
    0.9440, 0.9510, 0.9570, 0.9530, 0.9550, 0.9440, 0.9460, 0.9470,
0.9500,
    0.9530, 0.9500, 0.9520, 0.9520, 0.9550, 0.9500, 0.9530, 0.9550,
0.9520,
    0.9540, 0.9580, 0.9590, 0.9580, 0.9600, 0.9600, 0.9640, 0.9620,
0.9590,
    0.9640, 0.9640, 0.9660, 0.9620, 0.9610, 0.9650],
[0.1140, 0.8560, 0.8920, 0.9070, 0.9160, 0.9090, 0.9160, 0.9280,
0.9300,
    0.9360, 0.9410, 0.9430, 0.9460, 0.9450, 0.9390, 0.9490, 0.9510,
0.9500,
    0.9430, 0.9470, 0.9580, 0.9490, 0.9520, 0.9580, 0.9600, 0.9520,
0.9520,
    0.9490, 0.9490, 0.9530, 0.9560, 0.9560, 0.9610, 0.9600, 0.9620,
0.9590,
    0.9570, 0.9580, 0.9590, 0.9600, 0.9560, 0.9610, 0.9590, 0.9610,
0.9530,
    0.9570, 0.9580, 0.9560, 0.9550, 0.9580, 0.9570],
[0.0900, 0.8610, 0.8980, 0.9130, 0.9100, 0.9140, 0.9280, 0.9230,
0.9290,
    0.9310, 0.9410, 0.9370, 0.9500, 0.9440, 0.9460, 0.9460, 0.9510,
0.9520,
    0.9550, 0.9520, 0.9450, 0.9480, 0.9580, 0.9560, 0.9540, 0.9540,
0.9550,
    0.9500, 0.9610, 0.9600, 0.9560, 0.9550, 0.9560, 0.9590, 0.9580,
0.9600,
    0.9620, 0.9570, 0.9620, 0.9540, 0.9600, 0.9540, 0.9590, 0.9530,
0.9610,

```

```

0.9600, 0.9600, 0.9610, 0.9630, 0.9550, 0.9610],
[0.1200, 0.8530, 0.8910, 0.9050, 0.9170, 0.9220, 0.9260, 0.9360,
0.9330,
0.9330, 0.9410, 0.9440, 0.9360, 0.9450, 0.9410, 0.9510, 0.9420,
0.9510,
0.9430, 0.9520, 0.9470, 0.9540, 0.9520, 0.9480, 0.9470, 0.9520,
0.9560,
0.9550, 0.9570, 0.9480, 0.9570, 0.9530, 0.9550, 0.9590, 0.9600,
0.9530,
0.9530, 0.9560, 0.9610, 0.9620, 0.9580, 0.9640, 0.9690, 0.9630,
0.9600,
0.9610, 0.9670, 0.9650, 0.9540, 0.9540, 0.9650],
[0.0750, 0.8730, 0.8880, 0.8980, 0.9150, 0.9180, 0.9240, 0.9290,
0.9170,
0.9330, 0.9410, 0.9330, 0.9350, 0.9430, 0.9370, 0.9450, 0.9510,
0.9530,
0.9480, 0.9490, 0.9460, 0.9480, 0.9520, 0.9490, 0.9600, 0.9520,
0.9570,
0.9580, 0.9550, 0.9570, 0.9540, 0.9620, 0.9560, 0.9630, 0.9520,
0.9560,
0.9610, 0.9630, 0.9650, 0.9660, 0.9630, 0.9620, 0.9590, 0.9600,
0.9550,
0.9570, 0.9630, 0.9590, 0.9560, 0.9630, 0.9640]]),
'test_accs': tensor([[0.0926, 0.8768, 0.9053, 0.9197, 0.9266, 0.9296, 0.9322,
0.9335, 0.9397,
0.9405, 0.9412, 0.9458, 0.9489, 0.9507, 0.9489, 0.9523, 0.9536,
0.9508,
0.9554, 0.9548, 0.9554, 0.9589, 0.9612, 0.9603, 0.9605, 0.9590,
0.9615,
0.9620, 0.9612, 0.9640, 0.9624, 0.9633, 0.9626, 0.9624, 0.9632,
0.9623,
0.9641, 0.9645, 0.9643, 0.9641, 0.9647, 0.9660, 0.9659, 0.9647,
0.9652,
0.9666, 0.9653, 0.9670, 0.9675, 0.9678, 0.9683],
[0.1375, 0.8756, 0.9016, 0.9133, 0.9175, 0.9213, 0.9258, 0.9324,
0.9364,
0.9362, 0.9421, 0.9421, 0.9453, 0.9469, 0.9432, 0.9505, 0.9497,
0.9505,
0.9550, 0.9507, 0.9553, 0.9532, 0.9543, 0.9541, 0.9580, 0.9579,
0.9612,
0.9582, 0.9594, 0.9610, 0.9610, 0.9605, 0.9600, 0.9615, 0.9622,
0.9617,
0.9634, 0.9644, 0.9640, 0.9635, 0.9655, 0.9651, 0.9645, 0.9633,
0.9651,
0.9634, 0.9648, 0.9636, 0.9658, 0.9661, 0.9651],
[0.0990, 0.8680, 0.9005, 0.9198, 0.9170, 0.9230, 0.9314, 0.9337,
0.9349,

```

```

0.9367, 0.9437, 0.9405, 0.9488, 0.9483, 0.9474, 0.9497, 0.9506,
0.9532,
0.9504, 0.9537, 0.9553, 0.9532, 0.9559, 0.9603, 0.9562, 0.9564,
0.9573,
0.9565, 0.9615, 0.9594, 0.9617, 0.9602, 0.9625, 0.9616, 0.9625,
0.9638,
0.9634, 0.9633, 0.9641, 0.9620, 0.9620, 0.9622, 0.9647, 0.9658,
0.9653,
0.9646, 0.9645, 0.9647, 0.9668, 0.9655, 0.9644],
[0.1135, 0.8693, 0.8994, 0.9100, 0.9165, 0.9239, 0.9314, 0.9345,
0.9390,
0.9385, 0.9424, 0.9481, 0.9458, 0.9462, 0.9469, 0.9479, 0.9531,
0.9533,
0.9516, 0.9559, 0.9548, 0.9576, 0.9559, 0.9575, 0.9598, 0.9587,
0.9596,
0.9618, 0.9590, 0.9596, 0.9626, 0.9626, 0.9637, 0.9629, 0.9625,
0.9589,
0.9611, 0.9609, 0.9631, 0.9665, 0.9635, 0.9646, 0.9650, 0.9658,
0.9663,
0.9654, 0.9657, 0.9655, 0.9640, 0.9660, 0.9670],
[0.0634, 0.8770, 0.9031, 0.9113, 0.9173, 0.9254, 0.9308, 0.9369,
0.9288,
0.9386, 0.9444, 0.9434, 0.9466, 0.9473, 0.9453, 0.9492, 0.9508,
0.9550,
0.9536, 0.9543, 0.9545, 0.9522, 0.9570, 0.9589, 0.9565, 0.9562,
0.9579,
0.9582, 0.9629, 0.9618, 0.9619, 0.9603, 0.9611, 0.9616, 0.9604,
0.9643,
0.9655, 0.9650, 0.9643, 0.9656, 0.9659, 0.9667, 0.9674, 0.9652,
0.9651,
0.9672, 0.9677, 0.9682, 0.9655, 0.9676, 0.9680]]),
'val_mean': tensor([0.0958, 0.8638, 0.8938, 0.9066, 0.9162, 0.9172, 0.9252,
0.9284, 0.9276,
0.9330, 0.9392, 0.9388, 0.9424, 0.9450, 0.9412, 0.9470, 0.9482,
0.9502,
0.9466, 0.9502, 0.9506, 0.9504, 0.9538, 0.9510, 0.9534, 0.9514,
0.9540,
0.9530, 0.9544, 0.9540, 0.9550, 0.9562, 0.9556, 0.9588, 0.9574,
0.9560,
0.9574, 0.9584, 0.9612, 0.9600, 0.9594, 0.9602, 0.9620, 0.9598,
0.9576,
0.9598, 0.9624, 0.9614, 0.9580, 0.9582, 0.9624])),
'val_stderr': tensor([0.0090, 0.0046, 0.0022, 0.0025, 0.0021, 0.0026, 0.0027,
0.0022, 0.0027,
0.0008, 0.0018, 0.0021, 0.0029, 0.0008, 0.0016, 0.0013, 0.0018,
0.0014,
0.0023, 0.0010, 0.0028, 0.0013, 0.0012, 0.0026, 0.0030, 0.0012,

```

```

0.0013,
    0.0016, 0.0022, 0.0021, 0.0009, 0.0015, 0.0017, 0.0016, 0.0018,
0.0016,
    0.0018, 0.0012, 0.0011, 0.0020, 0.0012, 0.0017, 0.0020, 0.0018,
0.0015,
    0.0013, 0.0016, 0.0019, 0.0019, 0.0017, 0.0015]),
    'test_mean': tensor([0.1012, 0.8733, 0.9020, 0.9148, 0.9190, 0.9246, 0.9303,
0.9342, 0.9358,
    0.9381, 0.9428, 0.9440, 0.9471, 0.9479, 0.9463, 0.9499, 0.9516,
0.9526,
    0.9532, 0.9539, 0.9551, 0.9550, 0.9569, 0.9582, 0.9582, 0.9576,
0.9595,
    0.9593, 0.9608, 0.9612, 0.9619, 0.9614, 0.9620, 0.9620, 0.9622,
0.9622,
    0.9635, 0.9636, 0.9640, 0.9643, 0.9643, 0.9649, 0.9655, 0.9650,
0.9654,
    0.9654, 0.9656, 0.9658, 0.9659, 0.9666, 0.9666])),
    'test_stderr': tensor([0.0122, 0.0019, 0.0010, 0.0021, 0.0019, 0.0014, 0.0012,
0.0008, 0.0019,
    0.0008, 0.0006, 0.0013, 0.0008, 0.0008, 0.0010, 0.0007, 0.0008,
0.0008,
    0.0010, 0.0009, 0.0002, 0.0013, 0.0012, 0.0012, 0.0009, 0.0006,
0.0008,
    0.0011, 0.0007, 0.0008, 0.0003, 0.0007, 0.0006, 0.0003, 0.0005,
0.0010,
    0.0007, 0.0007, 0.0002, 0.0008, 0.0007, 0.0008, 0.0005, 0.0005,
0.0002,
    0.0007, 0.0006, 0.0008, 0.0006, 0.0005, 0.0008])),
    'entropy_lv': {'steps': tensor([ 0, 200, 400, 600, 800, 1000,
1200, 1400, 1600, 1800,
    2000, 2200, 2400, 2600, 2800, 3000, 3200, 3400, 3600, 3800,
    4000, 4200, 4400, 4600, 4800, 5000, 5200, 5400, 5600, 5800,
    6000, 6200, 6400, 6600, 6800, 7000, 7200, 7400, 7600, 7800,
    8000, 8200, 8400, 8600, 8800, 9000, 9200, 9400, 9600, 9800,
    10000])},
    'val_accs': tensor([[0.0800, 0.8650, 0.8980, 0.9190, 0.9320, 0.9330, 0.9340,
0.9400, 0.9430,
    0.9420, 0.9420, 0.9440, 0.9480, 0.9440, 0.9470, 0.9530, 0.9500,
0.9540,
    0.9630, 0.9550, 0.9550, 0.9550, 0.9600, 0.9640, 0.9600, 0.9620,
0.9570,
    0.9560, 0.9580, 0.9640, 0.9580, 0.9560, 0.9620, 0.9590, 0.9660,
0.9630,
    0.9670, 0.9620, 0.9640, 0.9640, 0.9630, 0.9650, 0.9670, 0.9670,
0.9710,
    0.9710, 0.9700, 0.9650, 0.9750, 0.9670, 0.9700],
    [0.1140, 0.8830, 0.8890, 0.9150, 0.9230, 0.9240, 0.9340, 0.9410,

```

0.9380,
 0.9460, 0.9510, 0.9490, 0.9500, 0.9590, 0.9610, 0.9580, 0.9610,
 0.9530,
 0.9560, 0.9600, 0.9600, 0.9650, 0.9600, 0.9650, 0.9630, 0.9630,
 0.9670,
 0.9680, 0.9670, 0.9640, 0.9680, 0.9610, 0.9630, 0.9620, 0.9650,
 0.9650,
 0.9680, 0.9640, 0.9680, 0.9650, 0.9700, 0.9650, 0.9690, 0.9700,
 0.9710,
 0.9660, 0.9710, 0.9700, 0.9680, 0.9710, 0.9690],
 [0.0900, 0.8780, 0.9050, 0.9170, 0.9240, 0.9290, 0.9350, 0.9480,
 0.9430,
 0.9500, 0.9530, 0.9500, 0.9510, 0.9540, 0.9610, 0.9620, 0.9600,
 0.9540,
 0.9650, 0.9620, 0.9640, 0.9610, 0.9620, 0.9620, 0.9580, 0.9600,
 0.9710,
 0.9640, 0.9660, 0.9650, 0.9680, 0.9670, 0.9650, 0.9680, 0.9670,
 0.9630,
 0.9670, 0.9630, 0.9630, 0.9710, 0.9660, 0.9680, 0.9690, 0.9700,
 0.9730,
 0.9690, 0.9650, 0.9720, 0.9720, 0.9650, 0.9700],
 [0.1200, 0.8630, 0.8960, 0.9130, 0.9230, 0.9290, 0.9410, 0.9400,
 0.9470,
 0.9410, 0.9460, 0.9470, 0.9490, 0.9510, 0.9520, 0.9590, 0.9570,
 0.9630,
 0.9630, 0.9600, 0.9520, 0.9660, 0.9660, 0.9570, 0.9580, 0.9670,
 0.9600,
 0.9610, 0.9700, 0.9620, 0.9640, 0.9680, 0.9640, 0.9650, 0.9650,
 0.9590,
 0.9720, 0.9660, 0.9650, 0.9670, 0.9600, 0.9650, 0.9650, 0.9600,
 0.9610,
 0.9630, 0.9690, 0.9650, 0.9680, 0.9650, 0.9620],
 [0.0750, 0.8730, 0.8930, 0.9180, 0.9310, 0.9220, 0.9330, 0.9430,
 0.9430,
 0.9470, 0.9450, 0.9470, 0.9520, 0.9480, 0.9510, 0.9490, 0.9560,
 0.9550,
 0.9500, 0.9510, 0.9540, 0.9590, 0.9580, 0.9520, 0.9530, 0.9520,
 0.9590,
 0.9550, 0.9570, 0.9590, 0.9640, 0.9620, 0.9600, 0.9630, 0.9590,
 0.9630,
 0.9660, 0.9620, 0.9680, 0.9610, 0.9640, 0.9690, 0.9670, 0.9640,
 0.9650,
 0.9680, 0.9660, 0.9650, 0.9640, 0.9690, 0.9620]]),
 'test_accs': tensor([[0.0926, 0.8764, 0.9103, 0.9252, 0.9342, 0.9381, 0.9424,
 0.9447, 0.9485,
 0.9479, 0.9507, 0.9540, 0.9553, 0.9559, 0.9579, 0.9607, 0.9608,
 0.9621,

0.9660, 0.9615, 0.9611, 0.9660, 0.9653, 0.9646, 0.9645, 0.9675, 0.9662,
 0.9710, 0.9663, 0.9655, 0.9685, 0.9693, 0.9689, 0.9692, 0.9676, 0.9712,
 0.9703, 0.9713, 0.9693, 0.9695, 0.9705, 0.9671, 0.9693, 0.9719, 0.9714,
 0.9709, 0.9723, 0.9722, 0.9714, 0.9726, 0.9716],
 [0.1375, 0.8906, 0.9054, 0.9203, 0.9243, 0.9341, 0.9383, 0.9417,
 0.9452, 0.9486, 0.9518, 0.9530, 0.9519, 0.9555, 0.9567, 0.9564, 0.9607,
 0.9597, 0.9608, 0.9605, 0.9601, 0.9637, 0.9656, 0.9654, 0.9634, 0.9648,
 0.9645, 0.9668, 0.9665, 0.9675, 0.9652, 0.9686, 0.9671, 0.9672, 0.9671,
 0.9679, 0.9681, 0.9677, 0.9665, 0.9679, 0.9687, 0.9678, 0.9686, 0.9692,
 0.9695, 0.9695, 0.9681, 0.9702, 0.9705, 0.9696, 0.9692],
 [0.0990, 0.8863, 0.9104, 0.9242, 0.9278, 0.9357, 0.9398, 0.9442,
 0.9469, 0.9521, 0.9532, 0.9547, 0.9560, 0.9591, 0.9602, 0.9630, 0.9611,
 0.9626, 0.9636, 0.9612, 0.9628, 0.9666, 0.9658, 0.9671, 0.9659, 0.9660,
 0.9651, 0.9670, 0.9674, 0.9672, 0.9684, 0.9672, 0.9673, 0.9676, 0.9690,
 0.9690, 0.9688, 0.9675, 0.9688, 0.9700, 0.9702, 0.9682, 0.9699, 0.9706,
 0.9697, 0.9687, 0.9689, 0.9706, 0.9711, 0.9706, 0.9715],
 [0.1135, 0.8762, 0.9042, 0.9231, 0.9326, 0.9377, 0.9434, 0.9456,
 0.9484, 0.9515, 0.9531, 0.9512, 0.9554, 0.9553, 0.9581, 0.9569, 0.9609,
 0.9632, 0.9614, 0.9626, 0.9627, 0.9640, 0.9663, 0.9657, 0.9653, 0.9661,
 0.9668, 0.9680, 0.9682, 0.9688, 0.9667, 0.9698, 0.9695, 0.9687, 0.9676,
 0.9704, 0.9701, 0.9715, 0.9702, 0.9703, 0.9710, 0.9712, 0.9719, 0.9715,
 0.9724, 0.9718, 0.9721, 0.9705, 0.9731, 0.9717, 0.9737],
 [0.0634, 0.8796, 0.9043, 0.9212, 0.9324, 0.9336, 0.9411, 0.9436,
 0.9476, 0.9493, 0.9503, 0.9530, 0.9571, 0.9544, 0.9601, 0.9604, 0.9593,
 0.9625, 0.9626, 0.9643, 0.9631, 0.9638, 0.9627, 0.9656, 0.9661, 0.9635,
 0.9649, 0.9679, 0.9666, 0.9662, 0.9666, 0.9681, 0.9682, 0.9699, 0.9675,

```

0.9685,
    0.9688, 0.9690, 0.9680, 0.9696, 0.9688, 0.9688, 0.9706, 0.9693,
0.9689,
    0.9706, 0.9705, 0.9711, 0.9706, 0.9722, 0.9690]]),
    'val_mean': tensor([0.0958, 0.8724, 0.8962, 0.9164, 0.9266, 0.9274, 0.9354,
0.9424, 0.9428,
    0.9452, 0.9474, 0.9474, 0.9500, 0.9512, 0.9544, 0.9562, 0.9568,
0.9558,
    0.9594, 0.9576, 0.9570, 0.9612, 0.9612, 0.9600, 0.9584, 0.9608,
0.9628,
    0.9608, 0.9636, 0.9628, 0.9644, 0.9628, 0.9628, 0.9634, 0.9644,
0.9626,
    0.9680, 0.9634, 0.9656, 0.9656, 0.9646, 0.9664, 0.9674, 0.9662,
0.9682,
    0.9674, 0.9682, 0.9674, 0.9694, 0.9674, 0.9666])),
    'val_stderr': tensor([0.0090, 0.0038, 0.0027, 0.0011, 0.0020, 0.0020, 0.0014,
0.0015, 0.0014,
    0.0017, 0.0020, 0.0010, 0.0007, 0.0026, 0.0028, 0.0023, 0.0019,
0.0018,
    0.0028, 0.0020, 0.0022, 0.0020, 0.0014, 0.0024, 0.0016, 0.0025,
0.0027,
    0.0024, 0.0026, 0.0011, 0.0018, 0.0022, 0.0009, 0.0015, 0.0014,
0.0010,
    0.0010, 0.0007, 0.0010, 0.0017, 0.0017, 0.0009, 0.0007, 0.0019,
0.0022,
    0.0014, 0.0012, 0.0015, 0.0019, 0.0012, 0.0019])),
    'test_mean': tensor([0.1012, 0.8818, 0.9069, 0.9228, 0.9303, 0.9358, 0.9410,
0.9440, 0.9473,
    0.9499, 0.9518, 0.9532, 0.9551, 0.9560, 0.9586, 0.9595, 0.9606,
0.9620,
    0.9620, 0.9619, 0.9629, 0.9647, 0.9650, 0.9657, 0.9656, 0.9653,
0.9655,
    0.9672, 0.9668, 0.9676, 0.9672, 0.9685, 0.9683, 0.9682, 0.9685,
0.9694,
    0.9694, 0.9690, 0.9686, 0.9697, 0.9692, 0.9691, 0.9706, 0.9704,
0.9702,
    0.9703, 0.9704, 0.9709, 0.9713, 0.9713, 0.9710])),
    'test_stderr': tensor([0.0122, 0.0029, 0.0014, 0.0009, 0.0018, 0.0009, 0.0009,
0.0007, 0.0006,
    0.0008, 0.0006, 0.0006, 0.0009, 0.0008, 0.0007, 0.0012, 0.0003,
0.0006,
    0.0005, 0.0007, 0.0009, 0.0006, 0.0006, 0.0004, 0.0007, 0.0005,
0.0004,
    0.0003, 0.0005, 0.0005, 0.0007, 0.0004, 0.0005, 0.0005, 0.0008,
0.0006,
    0.0006, 0.0007, 0.0006, 0.0005, 0.0007, 0.0006, 0.0006, 0.0005,
0.0006,

```

```

        0.0005, 0.0008, 0.0004, 0.0005, 0.0005, 0.0009]]},
'ce_1e3': {'steps': tensor([ 0, 200, 400, 600, 800, 1000, 1200,
1400, 1600, 1800,
2000, 2200, 2400, 2600, 2800, 3000, 3200, 3400, 3600, 3800,
4000, 4200, 4400, 4600, 4800, 5000, 5200, 5400, 5600, 5800,
6000, 6200, 6400, 6600, 6800, 7000, 7200, 7400, 7600, 7800,
8000, 8200, 8400, 8600, 8800, 9000, 9200, 9400, 9600, 9800,
10000])},
'val_accs': tensor([[0.0800, 0.8990, 0.9180, 0.9360, 0.9350, 0.9440, 0.9450,
0.9490, 0.9480,
0.9450, 0.9440, 0.9530, 0.9550, 0.9570, 0.9540, 0.9590, 0.9620,
0.9540,
0.9600, 0.9690, 0.9640, 0.9640, 0.9610, 0.9590, 0.9610, 0.9610,
0.9580,
0.9590, 0.9620, 0.9610, 0.9630, 0.9660, 0.9670, 0.9720, 0.9660,
0.9680,
0.9700, 0.9630, 0.9640, 0.9710, 0.9610, 0.9720, 0.9660, 0.9770,
0.9700,
0.9730, 0.9710, 0.9630, 0.9720, 0.9710, 0.9710],
[0.1140, 0.9010, 0.9130, 0.9240, 0.9340, 0.9380, 0.9390, 0.9420,
0.9430,
0.9510, 0.9480, 0.9420, 0.9510, 0.9530, 0.9550, 0.9530, 0.9610,
0.9540,
0.9520, 0.9530, 0.9630, 0.9660, 0.9620, 0.9620, 0.9570, 0.9660,
0.9660,
0.9650, 0.9690, 0.9750, 0.9670, 0.9740, 0.9750, 0.9690, 0.9710,
0.9690,
0.9720, 0.9720, 0.9650, 0.9810, 0.9750, 0.9730, 0.9710, 0.9780,
0.9780,
0.9710, 0.9780, 0.9760, 0.9760, 0.9740, 0.9740],
[0.0900, 0.9000, 0.9040, 0.9280, 0.9340, 0.9280, 0.9410, 0.9480,
0.9480,
0.9520, 0.9550, 0.9580, 0.9540, 0.9570, 0.9620, 0.9630, 0.9590,
0.9640,
0.9620, 0.9620, 0.9600, 0.9620, 0.9670, 0.9640, 0.9660, 0.9650,
0.9670,
0.9690, 0.9680, 0.9660, 0.9720, 0.9730, 0.9680, 0.9750, 0.9700,
0.9770,
0.9700, 0.9730, 0.9710, 0.9730, 0.9720, 0.9740, 0.9710, 0.9690,
0.9740,
0.9770, 0.9720, 0.9690, 0.9670, 0.9740, 0.9720],
[0.1200, 0.8890, 0.9060, 0.9160, 0.9310, 0.9420, 0.9470, 0.9410,
0.9490,
0.9470, 0.9520, 0.9560, 0.9530, 0.9590, 0.9590, 0.9580, 0.9610,
0.9590,
0.9620, 0.9600, 0.9590, 0.9550, 0.9590, 0.9650, 0.9620, 0.9700,
0.9590,
0.9590,

```

```

0.9640, 0.9610, 0.9700, 0.9630, 0.9700, 0.9740, 0.9670, 0.9680,
0.9740,
0.9730, 0.9740, 0.9730, 0.9710, 0.9720, 0.9730, 0.9740, 0.9710,
0.9750,
0.9700, 0.9680, 0.9760, 0.9720, 0.9740, 0.9700],
[0.0750, 0.9000, 0.9140, 0.9270, 0.9270, 0.9330, 0.9350, 0.9420,
0.9450,
0.9490, 0.9500, 0.9510, 0.9490, 0.9540, 0.9550, 0.9580, 0.9550,
0.9590,
0.9620, 0.9530, 0.9600, 0.9540, 0.9630, 0.9570, 0.9580, 0.9620,
0.9670,
0.9650, 0.9640, 0.9620, 0.9520, 0.9640, 0.9580, 0.9640, 0.9550,
0.9610,
0.9650, 0.9650, 0.9710, 0.9660, 0.9680, 0.9630, 0.9620, 0.9600,
0.9640,
0.9670, 0.9630, 0.9640, 0.9540, 0.9620, 0.9650]]),
'test_accs': tensor([[0.0926, 0.9038, 0.9207, 0.9345, 0.9408, 0.9397, 0.9482,
0.9479, 0.9532,
0.9562, 0.9581, 0.9603, 0.9605, 0.9624, 0.9632, 0.9642, 0.9651,
0.9634,
0.9674, 0.9673, 0.9656, 0.9663, 0.9672, 0.9675, 0.9670, 0.9634,
0.9701,
0.9681, 0.9698, 0.9684, 0.9718, 0.9717, 0.9697, 0.9725, 0.9715,
0.9681,
0.9697, 0.9709, 0.9709, 0.9696, 0.9701, 0.9694, 0.9694, 0.9712,
0.9723,
0.9701, 0.9716, 0.9710, 0.9717, 0.9727, 0.9720],
[0.1375, 0.9010, 0.9196, 0.9306, 0.9359, 0.9446, 0.9455, 0.9492,
0.9516,
0.9514, 0.9579, 0.9593, 0.9604, 0.9600, 0.9601, 0.9610, 0.9647,
0.9631,
0.9638, 0.9617, 0.9687, 0.9669, 0.9680, 0.9659, 0.9676, 0.9703,
0.9696,
0.9696, 0.9718, 0.9731, 0.9703, 0.9710, 0.9736, 0.9705, 0.9727,
0.9707,
0.9711, 0.9691, 0.9697, 0.9728, 0.9720, 0.9732, 0.9729, 0.9737,
0.9746,
0.9733, 0.9738, 0.9731, 0.9729, 0.9748, 0.9725],
[0.0990, 0.9018, 0.9168, 0.9299, 0.9372, 0.9415, 0.9473, 0.9505,
0.9534,
0.9578, 0.9617, 0.9624, 0.9609, 0.9615, 0.9668, 0.9644, 0.9650,
0.9691,
0.9684, 0.9687, 0.9646, 0.9664, 0.9700, 0.9719, 0.9705, 0.9689,
0.9711,
0.9716, 0.9705, 0.9724, 0.9714, 0.9712, 0.9699, 0.9722, 0.9725,
0.9723,
0.9733, 0.9746, 0.9716, 0.9728, 0.9724, 0.9736, 0.9740, 0.9707,

```

```

0.9717,
    0.9719, 0.9714, 0.9715, 0.9734, 0.9742, 0.9719],
    [0.1135, 0.8950, 0.9171, 0.9246, 0.9351, 0.9400, 0.9473, 0.9509,
0.9520,
    0.9556, 0.9570, 0.9576, 0.9592, 0.9605, 0.9626, 0.9661, 0.9655,
0.9664,
    0.9666, 0.9690, 0.9663, 0.9651, 0.9702, 0.9655, 0.9683, 0.9692,
0.9710,
    0.9711, 0.9700, 0.9715, 0.9702, 0.9704, 0.9714, 0.9677, 0.9707,
0.9724,
    0.9710, 0.9722, 0.9734, 0.9745, 0.9724, 0.9721, 0.9714, 0.9703,
0.9727,
    0.9712, 0.9720, 0.9717, 0.9732, 0.9729, 0.9735],
    [0.0634, 0.8958, 0.9194, 0.9296, 0.9341, 0.9421, 0.9465, 0.9447,
0.9491,
    0.9565, 0.9569, 0.9552, 0.9618, 0.9618, 0.9636, 0.9623, 0.9637,
0.9661,
    0.9666, 0.9644, 0.9687, 0.9675, 0.9645, 0.9700, 0.9666, 0.9686,
0.9667,
    0.9723, 0.9695, 0.9695, 0.9679, 0.9714, 0.9713, 0.9728, 0.9646,
0.9695,
    0.9688, 0.9696, 0.9729, 0.9708, 0.9717, 0.9668, 0.9714, 0.9725,
0.9702,
    0.9714, 0.9733, 0.9742, 0.9686, 0.9726, 0.9735]]),
    'val_mean': tensor([0.0958, 0.8978, 0.9110, 0.9262, 0.9322, 0.9370, 0.9414,
0.9444, 0.9466,
    0.9488, 0.9498, 0.9520, 0.9524, 0.9560, 0.9570, 0.9582, 0.9596,
0.9580,
    0.9596, 0.9594, 0.9612, 0.9602, 0.9624, 0.9614, 0.9608, 0.9648,
0.9634,
    0.9644, 0.9648, 0.9668, 0.9634, 0.9694, 0.9684, 0.9694, 0.9660,
0.9698,
    0.9700, 0.9694, 0.9688, 0.9724, 0.9696, 0.9710, 0.9688, 0.9710,
0.9722,
    0.9716, 0.9704, 0.9696, 0.9682, 0.9710, 0.9704]),
    'val_stderr': tensor([0.0090, 0.0022, 0.0026, 0.0032, 0.0015, 0.0029, 0.0021,
0.0017, 0.0011,
    0.0013, 0.0019, 0.0028, 0.0011, 0.0011, 0.0015, 0.0016, 0.0012,
0.0019,
    0.0019, 0.0030, 0.0010, 0.0024, 0.0013, 0.0015, 0.0016, 0.0016,
0.0020,
    0.0016, 0.0016, 0.0026, 0.0033, 0.0019, 0.0030, 0.0019, 0.0029,
0.0027,
    0.0014, 0.0022, 0.0018, 0.0024, 0.0024, 0.0020, 0.0021, 0.0032,
0.0024,
    0.0017, 0.0025, 0.0028, 0.0038, 0.0023, 0.0015]),
    'test_mean': tensor([0.1012, 0.8995, 0.9187, 0.9298, 0.9366, 0.9416, 0.9470,

```

```

0.9486, 0.9519,
    0.9555, 0.9583, 0.9590, 0.9606, 0.9612, 0.9633, 0.9636, 0.9648,
0.9656,
    0.9666, 0.9662, 0.9668, 0.9664, 0.9680, 0.9682, 0.9680, 0.9681,
0.9697,
    0.9705, 0.9703, 0.9710, 0.9703, 0.9711, 0.9712, 0.9711, 0.9704,
0.9706,
    0.9708, 0.9713, 0.9717, 0.9721, 0.9717, 0.9710, 0.9718, 0.9717,
0.9723,
    0.9716, 0.9724, 0.9723, 0.9720, 0.9734, 0.9727]),
    'test_stderr': tensor([0.0122, 0.0017, 0.0008, 0.0016, 0.0012, 0.0009, 0.0005,
0.0011, 0.0008,
    0.0011, 0.0009, 0.0012, 0.0004, 0.0004, 0.0011, 0.0009, 0.0003,
0.0011,
    0.0008, 0.0014, 0.0008, 0.0004, 0.0010, 0.0012, 0.0007, 0.0012,
0.0008,
    0.0008, 0.0004, 0.0009, 0.0007, 0.0002, 0.0007, 0.0009, 0.0015,
0.0008,
    0.0008, 0.0010, 0.0007, 0.0009, 0.0004, 0.0013, 0.0008, 0.0006,
0.0007,
    0.0005, 0.0005, 0.0006, 0.0009, 0.0004, 0.0003]))}

```

[]: